RTOS Architecture

Tasks, scheduling, channels, Safety and supervision – Stage 4 design (WP7 + WP10)

Revision 1 – 25 October 2026. Companion to *Firmware Architecture Baseline* Rev 3 (22 October 2026).

Item	Value
Purpose	Explain, in implementation detail, how the firmware uses FreeRTOS: which tasks exist, on which core and at which priority, how the one-second schedule is executed, how data and alarms move between tasks, how Safety drives the grow-lamp relay, and how stalls are detected. It is the reference for implementing and reviewing WP7 and WP10.
Readers	Every team member who writes or reviews firmware. Section 2 is a FreeRTOS primer for readers new to it; the rest assumes it.
Authority	Firmware Architecture Rev 3 remains the design authority. This document refines its sections 8–12 and 18 for Stage 4 and cites them as §n. Where the two differ, the difference is stated and explained here (for example the analog filter windows in section 6.3).
Status	**This is a design, mostly not yet implemented.** Every task, file and channel carries a status tag: **[IMPLEMENTED]** exists in the code today, **[STAGE 4]** is built in this milestone, **[LATER]** belongs to WP8 or WP9, **[PLANNED NAME]** is the intended name of a file or symbol that does not exist yet.
Numbers	Measured values are labelled *measured* and come from firmware/README.md bring-up records. Everything else is a design budget or an acceptance target, not a measurement.
Target	ESP32-S3-WROOM-1-N16R8, PlatformIO espressif32@6.12.0, Arduino-ESP32 2.0.17 (ESP-IDF 4.4 based), project code in C.

Change log

Revision	Changes
Rev 1	First issue. Written at the start of Stage 4, after the six sensor drivers (WP1–WP6) were completed and tested on hardware.
Rev 1.1	Stage 4 implemented and run on S3 hardware (25 Oct 2026): DS18B20 timing corrected to start-then-read in the same frame (section 6.3); logger record 128 characters; file map tags updated. Most tags in sections 1–13 still read STAGE 4 = built in this milestone; it is now built. Measured results (schedule, lamp, stall tests, stacks, heap) are in firmware/README.md "Production run".

Contents

1. The design on one page . . . 3
2. FreeRTOS on the ESP32-S3 – what you need to know . . . 5
3. Platform facts this design relies on . . . 8
4. Task inventory . . . 9
5. Boot sequence . . . 11
6. The acquisition engine (AcqNormal) . . . 13
7. Safety and the grow-lamp relay . . . 17
8. Channels between tasks . . . 21
9. Interrupts . . . 22
10. Timing, latency and CPU load . . . 23
11. Logging, supervision and watchdogs (WP10) . . . 25
12. Memory . . . 28
13. Failure behaviour . . . 29
14. What changes in WP8 and WP9 . . . 30
15. Implementation plan and file map . . . 32
16. How we prove it works . . . 34
17. Rules for everyone writing firmware . . . 35

1. The design on one page

IN SHORT

Four application tasks run in Stage 4. **AcqNormal** (core 1) is the only code that touches the sensors: it runs every driver on an absolute-deadline schedule – a one-second frame, plus the BME280 every 250 ms – and publishes a copied snapshot after every completed reading. **Safety** (core 0, highest application priority) is the only code that drives the relay: it wakes on an alarm event or every 100 ms, latches alarms, and switches the grow lamp from the light reading – OFF whenever that reading is stale or faulty. **SerialLogger** and **Supervisor** make the system observable and catch stalls without ever blocking the first two.

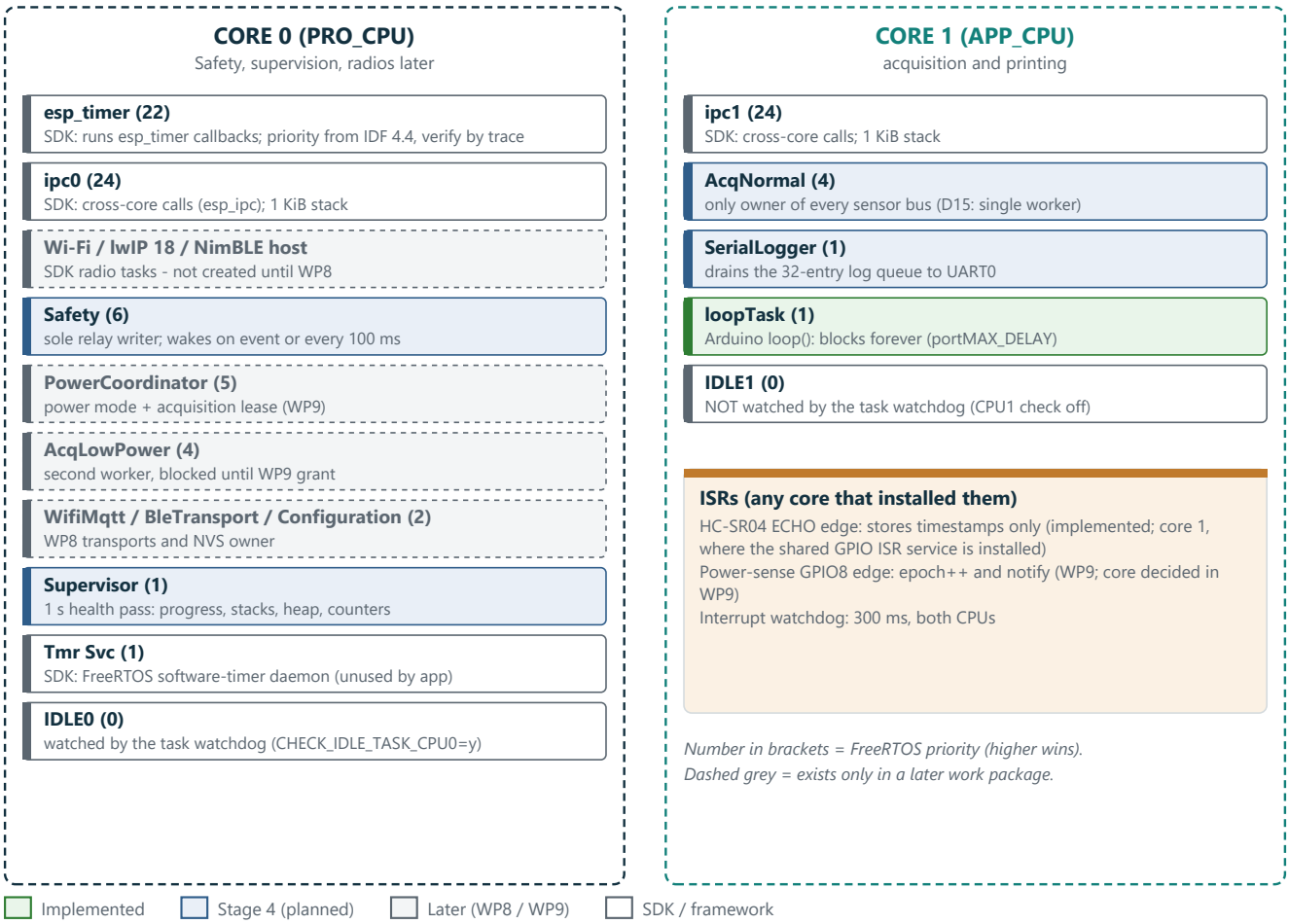


Figure 1 – Tasks per core after Stage 4. Blue = built in this milestone; dashed grey = later work packages.

The eight rules everything else follows from

#	Rule	Consequence in code
1	One owner per resource.	Only AcqNormal calls drivers or bus HALs. Only Safety writes GPIO15. Only SerialLogger writes the UART. No mutex is needed around any of them.
2	Readers never wait for a sensor.	The latest values live in a length-1 queue that the writer overwrites and readers peek with a zero timeout.
3	Every wait is bounded or event-driven.	Tasks block on a task notification with a timeout. Nothing spins, nothing sleeps a full second, nothing waits on the network.
4	Copies, not pointers, cross task boundaries.	Queues carry fixed-size records by value (Reading 56 B, Snapshot 584 B). No pointer into another task's memory is ever shared.
5	Alarms are sticky; detail is best-effort.	A per-rule sticky slot survives a full detail queue; the queue only adds history and counts what it dropped.
6	Stale data is unsafe data.	Safety re-checks reading age on every 100 ms wake, so a silent sensor or a stalled task turns the lamp OFF.
7	Decisions are pure functions.	Schedule, alarm and Safety logic live in src/domain (no FreeRTOS headers) and are unit-tested on the PC with injected time.
8	Allocate once, at boot.	Every task stack, queue and buffer is static. Nothing calls malloc after setup().

What exists today and what this milestone adds

Area	Today	After Stage 4
Drivers + HAL	[IMPLEMENTED] All seven drivers (BME280, DS18B20, DFR0026, HC-SR04, pH, TDS, SGP30), I2C, 1-Wire, ADC, pulse-capture ISR. Each runs in its own bring-up env.	Unchanged, plus the audit fixes listed in section 15.
Production main.c	[IMPLEMENTED] Applies (empty) safe defaults, logs the board gate, creates the snapshot queue, returns.	[STAGE 4] Starts Safety, AcqNormal, Supervisor and SerialLogger in a defined order.
Scheduling	[IMPLEMENTED] Bring-up harness: one driver per task, vTaskDelayUntil. test_water / test_climate interleave two drivers.	[STAGE 4] One multi-rate engine for all seven drivers with absolute deadlines.
Safety / relay	[IMPLEMENTED] domain/lamp_control decides; test_lamp drives the relay itself.	[STAGE 4] Safety task owns the relay and runs the lamp policy with staleness → OFF.
Logging	[IMPLEMENTED] logPrintf() calls printf directly (blocking).	[STAGE 4] 32-entry queue, SerialLogger task, drop-and-count.
Supervision	[IMPLEMENTED] Stubs only.	[STAGE 4] Progress deadlines, stack / heap checks, task watchdog subscriptions.
Power modes, transports	[IMPLEMENTED] rail_state record with portMUX; power sense measured on GPIO8.	[LATER] WP9 lease handover and second worker; WP8 BLE / MQTT / NVS.

2. FreeRTOS on the ESP32-S3 – what you need to know

IN SHORT

FreeRTOS gives each task its own stack and runs the highest-priority task that is ready, preempting lower ones at once. The ESP32-S3 runs two FreeRTOS schedulers, one per core, over shared memory. Tasks spend almost all their time *Blocked* on a queue, a notification or a delay; that is what makes a 1 ms tick and two cores enough for everything here.

2.1 Tasks and their states

A task is a C function that never returns, with its own stack and a priority. The scheduler keeps every task in one of four states:

State	Meaning	Example in this firmware
Running	Executing on a core right now. At most one task per core.	AcqNormal while it reads the BME280.
Ready	Could run, but a higher- or equal-priority task holds the core.	Supervisor while Safety runs on core 0.
Blocked	Waiting for an event or a timeout. Uses no CPU at all.	Safety in ulTaskNotifyTake(..., 100 ms); AcqNormal between ADC taps.
Suspended	Removed from scheduling until explicitly resumed. Not used here.	–

The project's tasks are all created with `xTaskCreateStaticPinnedToCore()`: the stack and the task control block are static arrays, and the task is fixed to one core. A task function is an endless loop whose first statement is a blocking wait.

```
static StaticTask_t s_tcb;
static StackType_t  s_stack[3 * 1024];          // bytes on ESP-IDF (not words)

static void safetyTask(void* arg) {
  for (;;) {
    ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(CFG_SAFETY_HEALTH_CHECK_MS));  // Blocked here
    safetyPass(monoNowMs());                                              // short, bounded
  }
}

const TaskSpec* t = cfgTaskSpec(TASK_ID_SAFETY);  // name, core, priority, stack
xTaskCreateStaticPinnedToCore(safetyTask, t->name, t->stackBytes, NULL,
                              t->priority, s_stack, &s_tcb, t->core);
```

ESP-IDF STACK SIZES ARE IN BYTES

Vanilla FreeRTOS counts stack depth in words; ESP-IDF counts bytes. `cfgTaskSpec()` already stores bytes (3 * 1024 for Safety). Do not multiply by 4.

2.2 Priorities and preemption

Priorities run from 0 (idle) to 24 on this build; a higher number wins. When a higher-priority task becomes Ready – because its notification arrived or its timeout expired – it preempts the running task on that core **immediately**, not at the next tick. Tasks of equal priority on the same core share it in 1 ms time slices.

Priority is only meaningful *within a core*. Safety (6) on core 0 and AcqNormal (4) on core 1 never compete: each core has its own scheduler. What matters for Safety is which other tasks share core 0 – in Stage 4 only Supervisor (1) and

SDK tasks; from WP8 on also the Wi-Fi, lwIP (18) and NimBLE tasks, which outrank it (decision D14, section 10.3).

2.3 Two cores, pinned tasks

ESP-IDF FreeRTOS is symmetric multiprocessing: one kernel, two schedulers, shared RAM. A task can be pinned to core 0, pinned to core 1, or left unpinned (`tskNO_AFFINITY`). **Every project task is pinned** (decision D07): the timing of a pinned task is reproducible, an ISR installed by it stays on that core, and its state never migrates mid-operation.

Core	Convention on ESP32	Our use
Core 0 (PRO_CPU)	Wi-Fi, Bluetooth controller, lwIP and most SDK services.	Safety, Supervisor; later PowerCoordinator, transports, Configuration.
Core 1 (APP_CPU)	Arduino loopTask and application code.	AcqNormal and SerialLogger. In low power (WP9) their work stops and core 1 only idles.

2.4 Time: the tick and the monotonic clock

The scheduler tick is 1 kHz (`CONFIG_FREERTOS_HZ=1000`), so `pdMS_TO_TICKS(ms)` is exact and a timeout has 1 ms resolution. A timeout of N ticks wakes after between N−1 and N ms, because the current tick is already partly over: deadlines below 2 ms are not meaningful as delays.

All ages and deadlines use `monoNowMs()` (`esp_timer_get_time() / 1000`, a 64-bit microsecond counter that does not wrap in practice), never the tick count and never wall-clock time. The tick count is only used inside the conversion to a timeout.

2.5 Waiting: delays, notifications and queues

Primitive	What it does	Where we use it
vTaskDelay(n)	Block for n ticks from now. Drifts: the work before it adds to the period.	Only the bring-up harness inside a poll loop. Not in production tasks.
vTaskDelayUntil(&last, n)	Block until last + n ticks: an absolute period without drift. Cannot be woken early.	Bring-up harness. Rejected for the production engine (section 6.5).
ulTaskNotifyTake(clear, timeout)	Block until another task or an ISR calls xTaskNotifyGive / vTaskNotifyGiveFromISR, or until the timeout. The cheapest wake-up FreeRTOS has: no separate object, 4 bytes in the TCB.	AcqNormal (timeout = next deadline), Safety (timeout = 100 ms), SerialLogger and Supervisor.
xQueueSend / Receive	Copy a fixed-size item in or out of a bounded FIFO. Optional timeout when full / empty.	Alarm detail queue, log queue. Always with a zero timeout on the sending side.
xQueueOverwrite / Peek	Length-1 queue used as a mailbox: the writer replaces the single item; readers copy it without removing it.	Snapshot store, SafetyStatus mailbox.

WHY – COPY SEMANTICS

A FreeRTOS queue copies the item *into* its own storage on send and *out* on receive. After `xQueueOverwrite(q, &snap)` returns, the writer may change `snap` freely; a reader holds its own private copy. That is why no lock is needed around a Snapshot. The price is the copy time: 584 bytes is about 1–2 µs at 240 MHz, negligible next to any sensor operation.

2.6 Shared data between cores: portMUX critical sections

When two tasks – possibly on different cores – or a task and an ISR must update the same small record, the update is wrapped in a spinlock critical section:

```
static portMUX_TYPE s_mux = portMUX_INITIALIZER_UNLOCKED;

portENTER_CRITICAL(&s_mux);        // task side; _ISR variants inside interrupts
slot->tripped = true; slot->count++; if (v > slot->worst) slot->worst = v;
portEXIT_CRITICAL(&s_mux);
```

Inside the section interrupts are disabled on this core and the other core spins if it tries to take the same lock. So the section must be a few instructions long: **no I/O, no printf, no FreeRTOS call that can block, no loop over a sensor**. rail_state.c already uses this pattern for the {valid, epoch} pair [IMPLEMENTED].

RULE – VOLATILE IS NOT SYNCHRONISATION

`volatile` stops the compiler caching a variable in a register; it does not make a multi-field update atomic and does not order memory between cores. A 64-bit `MonoMs` is written as two 32-bit halves, so a reader on the other core can see a torn value. Use a portMUX section (progress stamps, sticky alarm slots) or a queue copy.

2.7 Interrupts

An ISR runs on the core that installed it, preempts every task there, and must finish in microseconds. In this design ISRs only record a timestamp or bump a counter and, if needed, notify a task with `vTaskNotifyGiveFromISR()` followed by `portYIELD_FROM_ISR()`.

ISR rule	Reason
Mark the handler and everything it calls IRAM_ATTR.	Keeps the handler fast and cache-independent. Note: Arduino 2.0.17 installs its GPIO ISR service without ESP_INTR_FLAG_IRAM (CONFIG_ARDUINO_ISR_IRAM not set), so GPIO interrupts are held off while the flash cache is disabled (NVS / OTA writes) and an edge is serviced late rather than crashing. Any interrupt registered as IRAM-safe must have all its code in IRAM (audit L3).
No printf, no I2C / ADC, no malloc, no blocking call.	None of them are ISR-safe; most take locks.
Read GPIO levels by register, not digitalRead().	digitalRead() lives in flash; pulse_capture.c already does this [IMPLEMENTED].
Only *FromISR APIs, then yield if a higher task woke.	Otherwise the woken task waits for the next tick.

2.8 Memory: static allocation, internal RAM and PSRAM

The N16R8 has about 512 KiB of internal SRAM and 8 MiB of PSRAM. In this framework build `malloc()` of more than 4 KiB may return PSRAM (`CONFIG_SPIRAM_USE_MALLOC=y`, `ALWAYSINTERNAL=4096`). PSRAM is slower, is unreachable while the flash cache is off, and must never hold a task stack or ISR data. Static arrays are always placed in internal RAM, so the rule *allocate statically at boot* also settles the PSRAM question (decision D16).

2.9 Watchdogs

Watchdog	What it catches	Configuration (this build)
Interrupt watchdog	An ISR or critical section that holds a core too long.	300 ms, both CPUs. Panic and reset.
Task watchdog (TWDT)	A subscribed task that has not called esp_task_wdt_reset() within the timeout.	5 s, panic. IDLE0 is subscribed by the SDK; IDLE1 is not.
Progress stamps (ours)	A task that is alive but stuck in a wait or a retry loop, before the TWDT fires.	Supervisor, every 1 s (section 11).

3. Platform facts this design relies on

IN SHORT

The values below were read from the resolved framework package (framework-arduinoespressif32 3.20017.241212, tools/sdk/esp32s3/sdkconfig) on 25 October 2026. The README build record of 4 October quotes the original ESP32 sdkconfig; where the two differ, this table is the S3 truth.

Setting	Value	What it means for us
CONFIG_FREERTOS_UNICORE	not set	Two cores, two schedulers. A single-core image would be a different build, not a runtime switch.
CONFIG_FREERTOS_HZ	1000	1 ms tick; pdMS_TO_TICKS is exact.
CONFIG_ESP32S3_DEFAULT_CPU_FREQ_MHZ	240	**The CPU runs at 240 MHz.** CFG_CPU_MHZ_NORMAL = 160 in app_config.h is not applied anywhere (audit D9); WP9 decides the frequency policy.
CONFIG_FREERTOS_SUPPORT_STATIC_ALLOCATION	y	xTaskCreateStaticPinnedToCore, xQueueCreateStatic available.
CONFIG_FREERTOS_CHECK_STACKOVERFLOW	CANARY	Overflow detected at context switch by a canary word; reset with a 'stack overflow in task X' message.
CONFIG_FREERTOS_ISR_STACKSIZE	2096 B	Shared interrupt stack: keep ISRs shallow.
CONFIG_FREERTOS_IDLE_TASK_STACKSIZE	1024 B	No idle hooks in this project.
CONFIG_FREERTOS_TIMER_TASK_PRIORITY	1 (stack 3120 B)	Software-timer daemon exists but is unused by the application.
CONFIG_ESP_TASK_WDT_TIMEOUT_S	5	Equals CFG_TASK_WATCHDOG_MS. The Arduino core already initialises the TWDT; we only add our tasks.
CONFIG_ESP_TASK_WDT_CHECK_IDLE_TASK_CPU0 / CPU1	y / not set	Starvation of core 0 is caught by the SDK; starvation of core 1 is **not** – our progress stamps cover AcqNormal instead.
CONFIG_ESP_INT_WDT_TIMEOUT_MS	300	Any critical section or ISR longer than 300 ms resets the chip.
CONFIG_ARDUINO_RUNNING_CORE	1 (stack 8192 B)	loopTask runs setup() and loop() on core 1, priority 1. Our loop() blocks forever.
CONFIG_ESP_MAIN_TASK_AFFINITY	CPU0	app_main starts on core 0 and exits after creating loopTask.
CONFIG_ESP_IPC_TASK_STACK_SIZE	1024 B	ipc0 / ipc1 tasks at priority 24 run cross-core calls (e.g. installing an interrupt on the other core).
CONFIG_ESP_TIMER_TASK_STACK_SIZE	4096 B	esp_timer callbacks run in a high-priority task (22 in IDF 4.4 – verify by trace); we only read esp_timer_get_time().
CONFIG_LWIP_TCPIP_TASK_PRIO / AFFINITY	18 / CPU0	From WP8: the TCP/IP task outranks Safety on core 0 (D14).
CONFIG_ESP32_WIFI_TASK_PINNED_TO_CORE_0, BT_CTRL_PINNED_TO_CORE	y / 0	Radio work stays on core 0. NimBLE-Arduino pins its host task to core 0 (nimconfig.h).
CONFIG_SPIRAM_USE_MALLOC, ALWAYSINTERNAL	y, 4096	Large malloc may land in PSRAM – one more reason for static buffers.
CONFIG_PM_ENABLE	not set	No automatic frequency scaling or light sleep with this framework (§15).

Setting	Value	What it means for us
CONFIG_ESP_CONSOLE_UART_NUM / BAUDRATE	0 / 115200	printf goes to UART0, mirrored to USB-Serial-JTAG; lines end CRLF.

PlatformIO's `memory_type = qio_opi` selects the octal-PSRAM variant of the SDK headers at build time; the base sdkconfig above shows quad PSRAM, which does not apply to our board. None of the RTOS settings listed differ between the variants.

> **RE-CHECK AFTER ANY FRAMEWORK CHANGE**
> These values change with the platform version. After a platform bump, re-read the S3 sdkconfig and update this table before relying on the timing in later sections.

4. Task inventory

> **IN SHORT**
> Every task that will exist on the chip, ours and the SDK's, with core, priority, stack, trigger and status. In Stage 4 the application adds four tasks; five more exist in the architecture for WP8 and WP9. The table in app_config.h (cfgTaskSpec) is the single source of truth for names, cores, priorities and stack sizes – code never hard-codes them.

4.1 Application tasks

Task	Core / prio	Stack	Wakes on	Owns	Status
Safety	0 / 6	3 KiB	notification from acquisition, or 100 ms timeout	relay GPIO15, alarm latches, SafetyStatus	[STAGE 4]
AcqNormal	1 / 4	4 KiB	notification timeout = next job deadline	all sensor buses and drivers, the snapshot writer	[STAGE 4]
SerialLogger	1 / 1	2 KiB	log queue item (blocks on receive)	UART0 output	[STAGE 4]
Supervisor	0 / 1	3 KiB	1 s timeout	health checks, degraded status	[STAGE 4]
PowerCoordinator	0 / 5	4 KiB	power-sense ISR, transition ACKs	power mode, acquisition lease	[LATER] WP9
AcqLowPower	0 / 4	4 KiB	lease grant	3.3 V sensors in low power	[LATER] WP9
WifiMqtt	0 / 2	6 KiB	connection events, backoff	Wi-Fi + MQTT client	[LATER] WP8
BleTransport	0 / 2	6 KiB	RX queue, telemetry deadlines	GATT sends	[LATER] WP8
Configuration	0 / 2	4 KiB	command queue	NVS, registry	[LATER] WP8

4.2 Framework and SDK tasks that share the cores

Task	Core / prio	Role	Effect on us
loopTask	1 / 1	Runs setup() then loop(). Our loop() blocks forever.	8 KiB stack wasted but harmless; setup() is where we create our tasks.
IDLE0, IDLE1	0 / 0, 1 / 0	Run when nothing else is Ready; free deleted-task memory.	Must get CPU time: IDLE0 is watched by the TWDT.
ipc0, ipc1	0 / 24, 1 / 24	Execute functions on a specific core on request.	Brief; used by the SDK e.g. for interrupt allocation.
esp_timer	0 / 22 (verify)	Dispatches esp_timer callbacks.	We use esp_timer only as a clock, so it stays idle.
Tmr Svc	0 / 1	FreeRTOS software timers.	Unused by the application.
Wi-Fi, lwIP (18), NimBLE host	0 / high	Radio and network stacks.	[LATER] Absent until WP8; then they preempt Safety (D14).

4.3 Why these priorities

- **Safety is highest (6)** because its work is the shortest and the most urgent: when an alarm event arrives it must act within 10 ms (§19). Being highest on core 0 among application tasks means only SDK tasks and ISRs can delay it.
- **AcqNormal (4) is alone at its level on core 1**; the only other core-1 tasks are SerialLogger and loopTask at 1. So printing can never delay a sensor deadline: the logger only runs while acquisition is Blocked.
- **Supervisor and SerialLogger are lowest (1)**: they observe and report, and must never steal time from the tasks they observe. If they starve, the system still works – only reporting is late.
- **Equal priorities are avoided on a core** wherever possible, so no task depends on time-slicing to make progress.

4.4 Why the work is split across cores this way

Acquisition is the heaviest and most timing-sensitive application work and needs no radio, so it gets core 1 almost to itself. Safety stays on core 0 in every power mode, so the relay controller never migrates when WP9 moves sensing to core 0 in low power (§8). The cost is that from WP8 on Safety shares core 0 with the radio stacks; the ≤ 10 ms target is therefore re-tested under radio load, with a documented fallback (section 10.3).

5. Boot sequence

IN SHORT

setup() runs once in loopTask on core 1. It makes the outputs safe first, then creates the static queues, then starts Safety before acquisition so the first alarm always has a receiver. A failed step is logged and the boot continues; the relay is already de-energised and only Safety could change that.

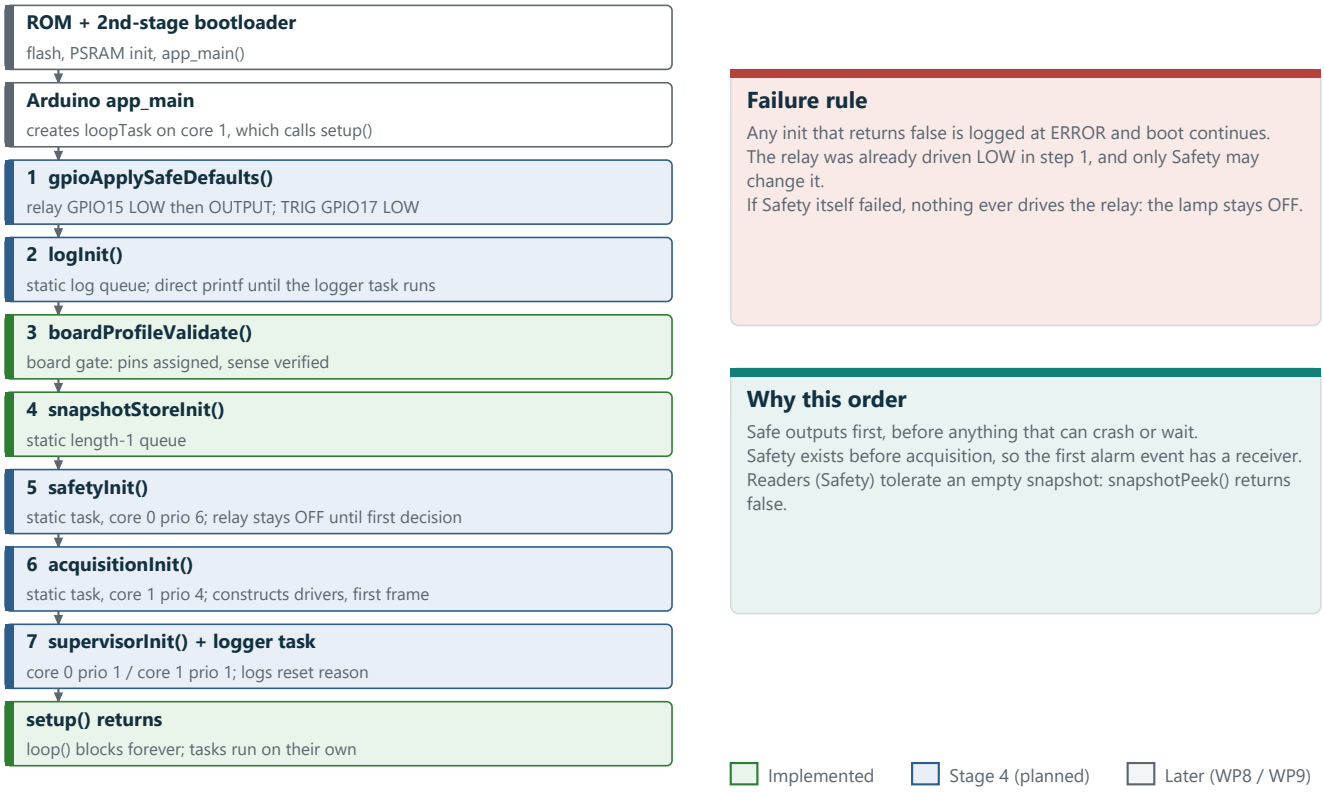


Figure 2 – setup() order after Stage 4. Function names are the existing public APIs in services/ and diagnostics/.

Step details

Step	What happens	Status
1 Safe outputs	gpioApplySafeDefaults() writes GPIO15 LOW *before* making it an output, so the pin never glitches high; then GPIO17 (HC-SR04 TRIG) LOW. The 10 kΩ gate pull-down already holds the relay off from reset until this line.	[STAGE 4] fills the empty stub (audit L4)
2 Logger	logInit() creates the static log queue. Until the SerialLogger task exists, logPrintf() prints directly, so early boot messages are never lost.	[STAGE 4]
3 Board gate	boardProfileValidate(): pins assigned, power sense verified. Printed as 'board gate', not 'production ready' (audit D2).	[IMPLEMENTED] (wording [STAGE 4])
4 Snapshot	snapshotStoreInit(): static length-1 queue of one Snapshot.	[IMPLEMENTED]
5 Safety	safetyInit(): sticky store, detail queue, SafetyStatus mailbox, task on core 0. The relay stays OFF until the first lamp decision, which itself starts OFF and has a dwell.	[STAGE 4]
6 Acquisition	acquisitionInit(): construct all drivers and HALs, create AcqNormal on core 1. The first frame starts immediately; drivers that need warm-up report STARTING / WARMING.	[STAGE 4]
7 Supervisor + logger	Create SerialLogger (core 1) and Supervisor (core 0). Supervisor logs esp_reset_reason() once – the first line to read after an unexpected reset.	[STAGE 4]
Return	loop() blocks with vTaskDelay(portMAX_DELAY). No work is ever done in loop().	[IMPLEMENTED]

WORKED EXAMPLE – WHAT THE CONSOLE SHOWS AFTER A NORMAL BOOT (ILLUSTRATIVE)

```
[I] hydroponics firmware - stage 4
[I] reset reason: POWERON
[I] board gate: power sense yes, relay yes -> pass
[I] safety: started (core 0, prio 6), lamp auto ON, relay OFF
[I] acq: started (core 1, prio 4), 7 drivers
[I] ds18b20 init -> OK, ROM 28-00000098BB42
[I] sup: t=5 s | airT 23.1 RH 48 | waterT 21.4 | light 63 mV | pH UNCAL | TDS 466 | lamp OFF
```

6. The acquisition engine (AcqNormal)

IN SHORT

One task on core 1 owns all seven drivers. It keeps a table of jobs with absolute deadlines, sleeps until the earliest one, and on each wake advances every due driver by one bounded step. Slow work – conversions inside a sensor, filter windows of spaced ADC taps – happens while the task is Blocked, so a single task can run all sensors in parallel without threads or locks. Every completed reading is stamped, stored in the working snapshot, checked against its alarm rules and published at once.

6.1 One worker, one owner (D05, D15)

Until WP9 there is exactly one acquisition task. It constructs every driver and bus HAL (I2C controller 0 on GPIO21/47, 1-Wire on GPIO14, ADC1 channels on GPIO1/6/7, pulse capture on GPIO17/18) with the pins from board_profile.h, and no other task ever calls them. Because nothing else touches a bus, there is no mutex, no priority inversion, and no possibility of two transactions interleaving on the shared BME280 / SGP30 I2C bus. `acquisitionGrant()` and `acquisitionRequestQuiesce()` stay stubs until WP9 adds the second worker and the lease (section 13).

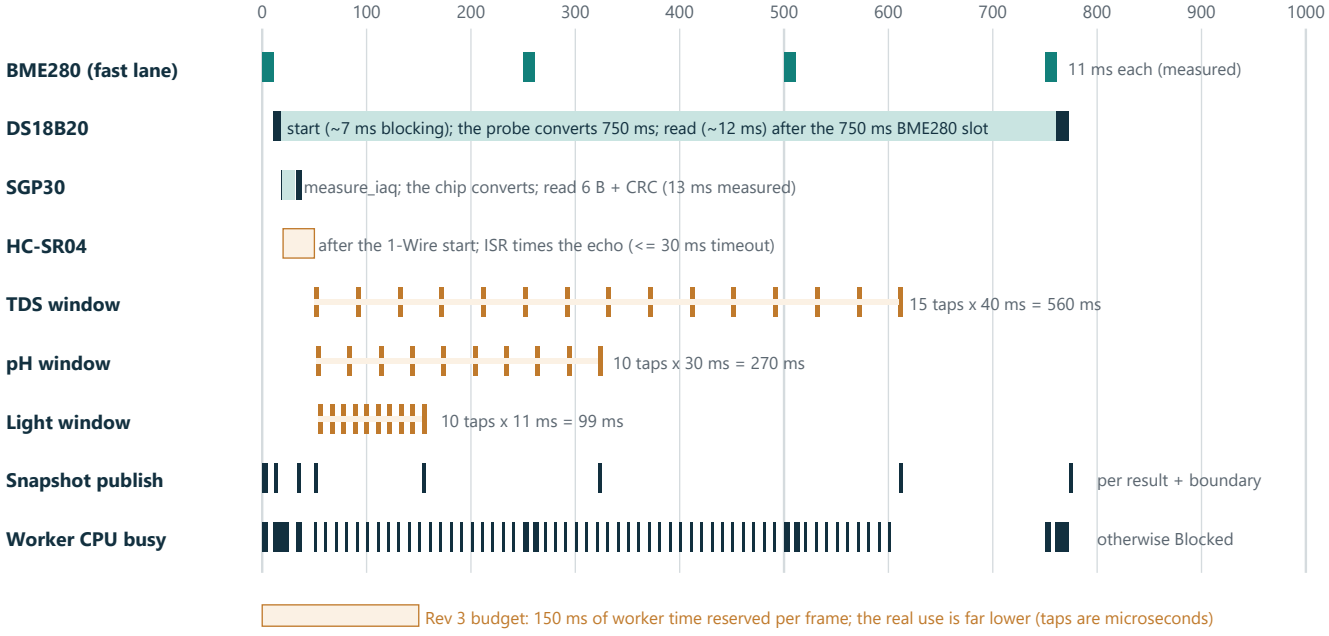
6.2 The job table

The schedule is data, not code: a small static table of jobs, each with an absolute next deadline. The table and its pure functions live in `src/domain/schedule.c` **[IMPLEMENTED]** so they are host-tested; the task only calls them.

Job	Period / phase	Driver steps	Host budget	Inputs from other sensors
BME280	250 ms / 0, 250, 500, 750	start (forced) → poll PENDING to +10 ms → READY	12 ms elapsed, < 1 ms CPU (11 ms measured)	–
DS18B20	1 s / first, after the BME280 slot	start (Convert T) → probe converts 750 ms → poll reads the scratchpad at ≈ +760 ms, same frame	~7 ms then ~12 ms blocking, separately	–
SGP30	1 s / after DS18B20	start (measure_iaq) → PENDING to +13 ms → READY	15 ms (13 ms measured)	absolute humidity from a fresh BME280 sample
HC-SR04	1 s / after the 1-Wire start	start (10 µs trigger) → ISR timestamps → READY / TIMEOUT	≤ 30 ms timeout	–
TDS	1 s / after HC-SR04 trigger	15 ADC taps, 40 ms apart, trimmed mean	≈ 560 ms elapsed, µs per tap	latest DS18B20 water temperature (0.2–1.2 s old)
pH	1 s / same	10 taps, 30 ms apart	≈ 270 ms elapsed	–
Light	1 s / same	10 taps, 11 ms apart	≈ 99 ms elapsed	–
Frame boundary	1 s / 0	record jitter, publish snapshot, re-arm the 1 s jobs	≤ 10 ms (Rev 3 budget)	–

A job has three timing fields: `periodMs`, `nextDueMs` (absolute, monotonic) and `budgetMs`. A driver that is PENDING additionally carries the `nextPollMs` it returned. The task's next wake-up is the minimum over all of them.

6.3 One second, step by step



Planned interleave (Stage 4), drawn from driver constants and measured bring-up times - not yet a trace.

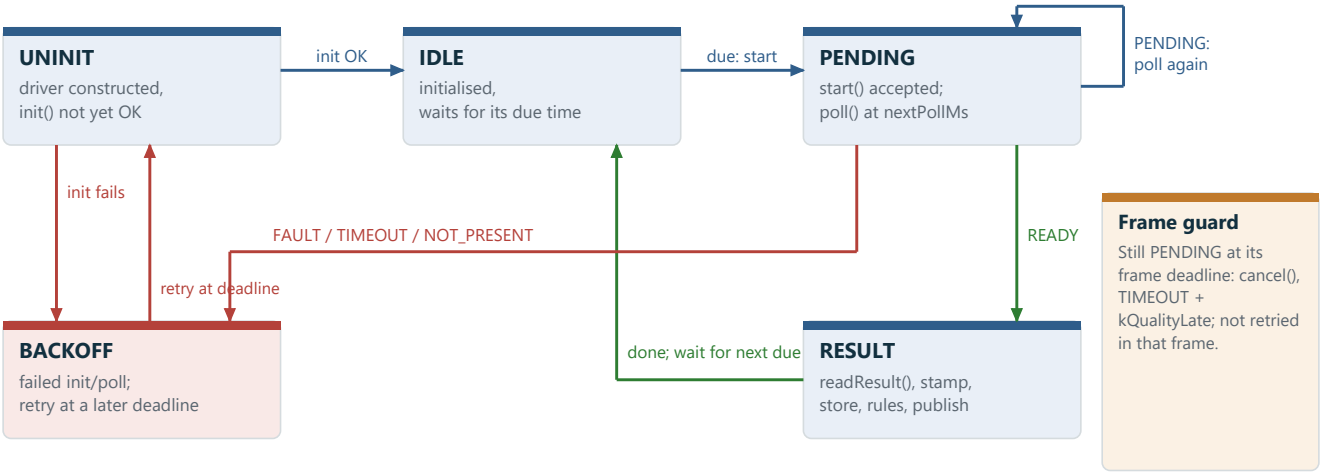
Figure 3 – Planned interleave of one frame. Dark = the worker is running; light = the sensor converts on its own; amber ticks = single ADC conversions.

- **t = 0.** Frame boundary: the worker records how late it woke (release jitter), publishes the snapshot of the previous frame and re-arms the one-second jobs. The BME280 slot is also due, so its forced conversion starts now.
- **t ≈ 10 ms.** BME280 READY (11 ms measured): result stored, rule check, publish.
- **t ≈ 11–18 ms.** DS18B20 start: reset, Match ROM, Convert T (~7 ms blocking). It waits until the BME280 conversion is done, because a blocking call during it would make the BME280 poll late and time out. The probe now converts on its own for 750 ms.
- **t ≈ 18 ms.** SGP30 measure command (with humidity compensation from the fresh BME280 reading); the chip converts while the worker continues.
- **t ≈ 20 ms.** HC-SR04 trigger – after the 1-Wire start, and only if the DS18B20 read is not due within the 30 ms echo window, because 1-Wire slots mask interrupts for up to 70 µs and would delay the echo-edge timestamp (onewire_bus.h).
- **t ≈ 32 ms.** SGP30 READY. **t ≈ 50 ms.** HC-SR04 READY or TIMEOUT.
- **t ≈ 50 ms onward.** The three analog windows start. Each tap is one ADC1 conversion of a few microseconds; between taps the worker is Blocked. Taps from different channels interleave freely because the worker performs them one at a time.
- **t = 250, 500, 750 ms.** BME280 slots preempt the order *between* steps: a blocking step whose budget would cross the slot, or that would run while a BME280 conversion is pending, is deferred until after it.
- **t ≈ 153 / 322 / 610 ms.** Light, pH and TDS windows complete; each result is published immediately.
- **t ≈ 761–773 ms.** DS18B20 read (~12 ms blocking), after the 750 ms slot's BME280 is done: CRC check, result published. TDS uses this value in the next frame, so its compensation input is 0.2–1.2 s old (limit 3 s). From then to 1000 ms the worker only idles.

REFINEMENT OF REV 3 SECTION 9

Rev 3 drew the analog group as 3 × 5 ms of CPU. The drivers written since use spaced filter windows (pH 270 ms, TDS 560 ms, light 99 ms elapsed), exactly as Rev 3's rule 'do not compress a spaced filter window into a burst' asks. CPU use stays far below the 150 ms reservation; elapsed time is what the schedule must fit, and it fits inside one frame.

6.4 Driving one driver: the DriverRun state machine



One DriverRun per driver. The same stepping is proven in test/bringup/water_main.c (beginRun / stepRun / cancelRun).

Figure 4 – Per-driver stepping. Every transition is one bounded driver call.

Situation	What the engine does	Reading published
init() fails	Back off: retry at a later frame (bounded, e.g. every 5 s), never in the same frame.	STARTING, or FAULT if it worked before
start() / poll() returns FAULT	Reading FAULT; DriverRun → IDLE. Next attempt at the next period.	FAULT, last-good kept separately
poll() returns NOT_PRESENT	Mark uninitialised; init again at the back-off deadline.	FAULT (audit L2 makes the DS18B20 do this)
Still PENDING at its frame deadline	cancel(); reading TIMEOUT with kQualityLate.	TIMEOUT
I2C NACK on init	Only a stuck bus (SDA held LOW) triggers i2cBusRecover(); a missing device just fails (audit L5).	FAULT
Dependency missing	TDS without fresh water temperature, SGP30 without fresh humidity: the driver reports it.	DEPENDENCY_INVALID / uncompensated flag

6.5 How the task waits

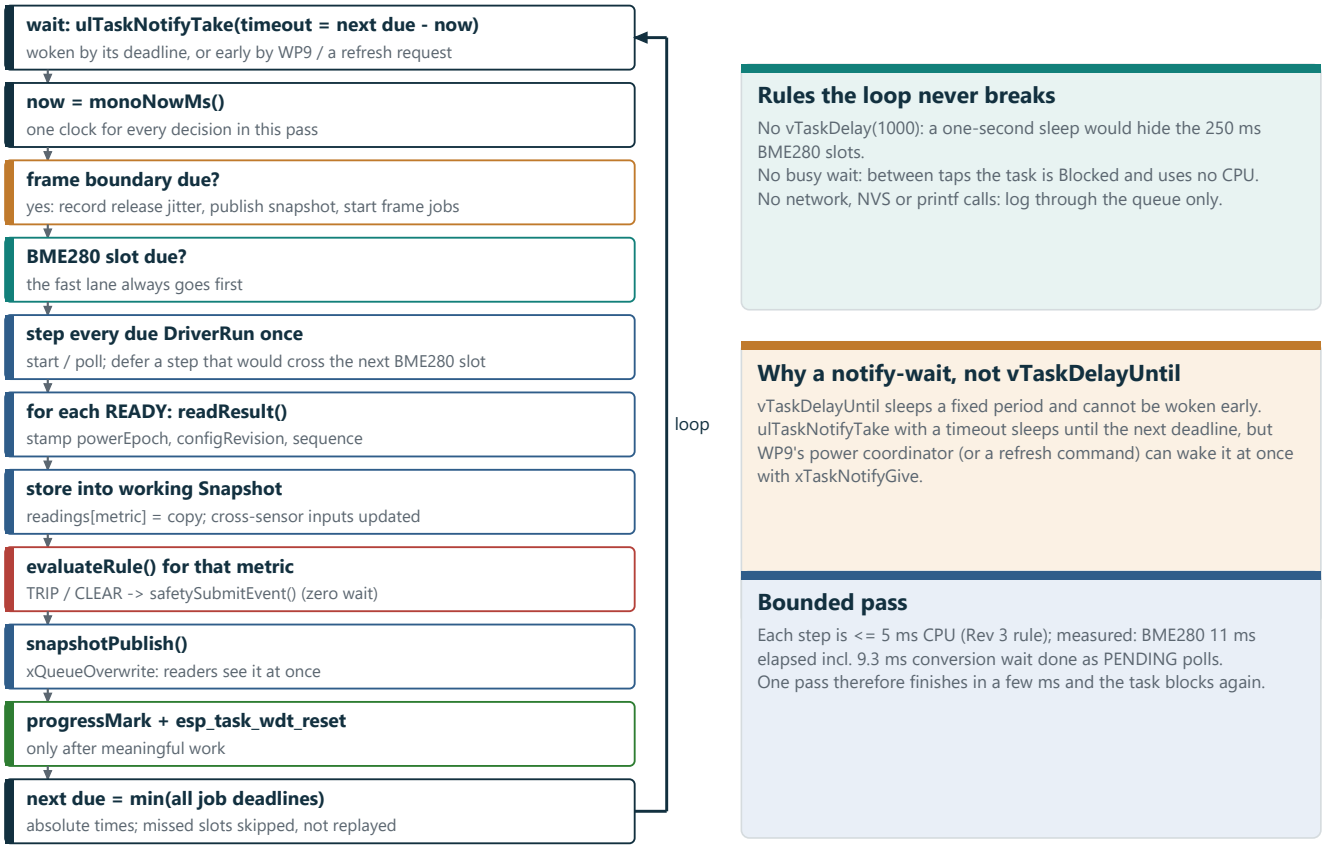


Figure 5 – AcqNormal's loop. One pass is a few milliseconds; the rest of the time the task is Blocked.

```
for (;;) {
  const MonoMs now0 = monoNowMs();
  const MonoMs due  = scheduleEarliestDue(s_jobs, JOB_COUNT);  // min of all deadlines
  ulTaskNotifyTake(pdTRUE, due > now0 ? pdMS_TO_TICKS(due - now0) : 0);
  const MonoMs now = monoNowMs();
  acqRunDueJobs(now);           // bounded: each job advances one step
  progressMark(TASK_ID_ACQUISITION_NORMAL);
  esp_task_wdt_reset();
}
```

6.6 Deadlines: skip, never catch up

When a job runs late – because a blocking DS18B20 call overran, or the task was starved – `scheduleJobMarkRun()` advances its deadline by **whole periods past now**. Missed slots are counted (`late`, `skipped`) and reported by Supervisor; they are never replayed in a burst. A burst would put several readings milliseconds apart, add no information and push every other deadline further out.

WORKED EXAMPLE – A 30 MS OVERRUN

The BME280 slot is due at 500 ms. The DS18B20 reset takes longer than budgeted and the worker reaches the slot at 530 ms. The BME280 runs once, at 530 ms, flagged late (30 ms). Its next deadline is 750 ms – not 780 ms, and not an extra catch-up run – so the 250 ms grid is kept. Supervisor's next line shows `late=1`.

6.7 After every completed reading

Step	Detail
1 Read	sensorDriverReadResult() copies up to 4 Reading records into a local array.
2 Stamp	powerEpoch (0 until WP9), configRevision, and the per-metric sequence counter (so readers can tell a new sample from a re-published one).
3 Cross-sensor	BME280 T + RH → sgp30 humidity input; DS18B20 water T → tdsDriverSetWaterTemperature(). The consumer driver checks the age itself.
4 Store	s_work.readings[metric] = reading. s_work is a static Snapshot, the only copy acquisition mutates.
5 Rules	For each enabled rule on that metric: evaluateRule(rule, &r, now, staleLimit, &state). TRIP or CLEAR → safetySubmitEvent() (zero wait) which also notifies Safety.
6 Publish	snapshotPublish(&s_work): xQueueOverwrite copies it; readers see the new value immediately.

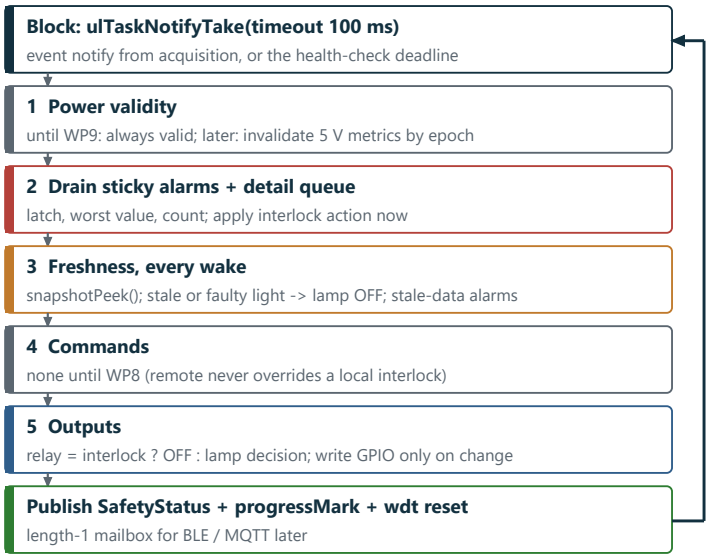
WHY – PUBLISH AFTER EVERY READING, NOT ONCE PER SECOND

A light or temperature value is visible to Safety a few microseconds after it is measured instead of up to a second later. The cost is one 584-byte copy per reading – about a dozen per second.

7. Safety and the grow-lamp relay

IN SHORT

Safety is a small state machine on core 0 at the highest application priority. It is the only code that writes GPIO15. It wakes when acquisition reports an alarm event, and in any case every 100 ms. Each wake runs the five steps of §12 in a fixed order, then blocks again. The grow lamp is its first automatic action: ON in low light, with hysteresis and a minimum dwell, and OFF whenever the light data is stale or faulty or an interlock is latched.



Why it wakes every 100 ms

An acquisition stall produces NO events. Only a timed wake can notice that the light reading has stopped ageing forward and switch the lamp OFF. (Fixes audit B1.)

Static memory (3 KiB stack)

The 584-byte Snapshot copy and the drained event buffer are static variables, not locals.
Safety never blocks on anything except its own notification: no mutex, no queue receive with a timeout, no NVS, no network.

Figure 6 – Safety's wake loop. Steps 1 and 4 are placeholders until WP9 and WP8.

7.1 Where the logic lives

The task in `services/safety.c` only waits, copies inputs and writes the pin. All decisions are in a pure function, `safetyCoreStep()` in `src/domain/safety_core.c` **[IMPLEMENTED]**:

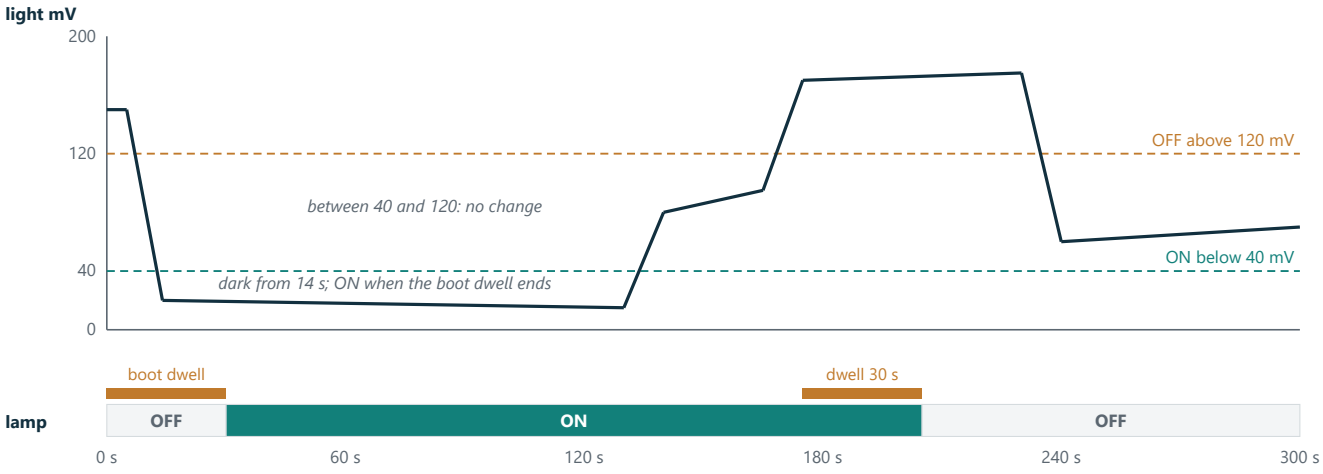
```
typedef struct SafetyInputs {
  const Snapshot*   snap;          // NULL until the first publish
  const AlarmEvent* events; uint8_t eventCount;
  MonoMs            now;
} SafetyInputs;

typedef struct SafetyOutputs {
  bool       relayOn;       // what relay 1 (GPIO15) must be
  bool       lampWantsOn;   // lamp decision before interlock / enable
  LampReason lampReason;    // level, dwell, fault, stale, interlock, ...
  bool       interlock;
  uint16_t   latchedMask, activeMask, staleMask;  // one bit per rule
} SafetyOutputs;

void safetyCoreStep(SafetyCore* self, const SafetyInputs* in, SafetyOutputs* out);
```

Because it takes `now` as an argument, every scenario in this section is a host unit test: build a snapshot, advance a fake clock, check the relay output.

7.2 The lamp policy



Placeholder thresholds and dwell from test_lamp; final values are measured at the sensor's final position.

Figure 7 – Hysteresis band and dwell. The lamp changes state only outside the band and only after the minimum time in the current state.

Rule	Behaviour	Source
Hysteresis	ON below POLICY_LAMP_ON_BELOW_MV (40), OFF above POLICY_LAMP_OFF_ABOVE_MV (120), unchanged in between.	domain/lamp_control **[IMPLEMENTED]**
Dwell	A state is held at least POLICY_LAMP_MIN_ON_MS / _MIN_OFF_MS (30 s bench placeholder; minutes in use). Boot counts as a switch to OFF, so a reset loop cannot flash the lamp.	**[IMPLEMENTED]**
Saturated	A clipped reading (kQualitySaturated) counts as bright.	**[IMPLEMENTED]**
Faults	POLICY_LAMP_FAULTS_TO_OFF unusable readings in a row force OFF at once. Counted per new light result (intervalEndMs changed – the FAULT path does not bump sequence), not per wake.	**[IMPLEMENTED]** + **[STAGE 4]**
Staleness	If the newest light reading ended more than CFG_STALE_ORDINARY_MS (3 s) ago, force OFF on this wake – no new sample needed.	**[STAGE 4]** fixes audit B1
Interlock	A latched alarm whose action is DEENERGIZE on relay 1 forces OFF and wins over the lamp.	**[STAGE 4]**
Enable switch	`POLICY_LAMP_AUTO_ENABLED` (default 1). With 0, Safety still computes and reports the decision but never energises the relay.	**[STAGE 4]**

WHY AGE COMES FROM INTERVALENDMS

The light driver publishes UNCALIBRATED millivolts. `readingHealthAt()` only ages readings whose health is OK, so it would never call an UNCALIBRATED light reading stale. Safety therefore computes the age itself as `now – reading.intervalEndMs`. (Audit D6 also makes `lastGoodMs` mean 'last usable result' for every driver.)

WORKED EXAMPLE – COVERING THE SENSOR AT NIGHT

1. The room is dark: light readings ~20 mV arrive once per second. On the first one below 40 mV, lamp_control wants ON, but the boot dwell has 12 s left: `waiting = true`.
2. Twelve seconds later the dwell expires; the next reading (still below 40 mV) switches the decision to ON. Safety writes GPIO15 HIGH; the relay closes NO; the lamp lights.
3. The lamp light now reaches the sensor: 95 mV – inside the band, no change. (If it read above 120 mV the lamp would turn itself off; the threshold must be set above the 'night + lamp ON' reading.)
4. Morning: 300 mV, above 120. After the 30 s ON dwell the decision becomes OFF and GPIO15 goes LOW.

WORKED EXAMPLE – THE LIGHT DRIVER STOPS PRODUCING RESULTS

The S wire is fine but the ADC driver begins returning FAULT on every window. Five FAULT readings in a row (5 s) force OFF through the fault counter. If instead AcqNormal itself hangs, no reading arrives at all: on every 100 ms wake Safety sees the light reading's age grow, and once it passes 3 s the lamp is forced OFF – without a single new event. Supervisor reports the AcqNormal stall at about the same time, and the task watchdog resets the chip after 5 s (section 11).

WHAT FIRMWARE CANNOT DETECT

If the S wire is pulled off, the 1 µF capacitor decays to ~0 mV, which reads as darkness: the lamp switches ON. No software rule can distinguish this from a real dark room. A pull-down or a plausibility rule (e.g. 0 mV in daytime hours) needs a hardware or requirements decision.

7.3 Alarm rules and latching

Acquisition evaluates rules (section 6.7); Safety latches and acts. A rule (`AlarmRule` in domain/alarm.h **[IMPLEMENTED]**) has a metric, comparison, trip and clear thresholds (hysteresis), persistence time, severity, latching flag and actuator action. The evaluator gains a time and state argument (audit L1):

```
RuleDecision evaluateRule(const AlarmRule* rule, const Reading* sample,
                          MonoMs now, uint32_t staleLimitMs, AlarmRuleState* state);
// NOT_ELIGIBLE unless readingIsActionableAt(sample, now, staleLimitMs)
// TRIP  when beyond trip threshold for >= persistMs (0 = first valid sample)
// CLEAR when back past the clear threshold (hysteresis)
```

Situation	Result
Valid sample beyond trip for persistMs	TRIP event → sticky slot set → Safety latches, applies the action (relay OFF if DEENERGIZE).
Later normal sample before Safety ran	The sticky slot keeps tripped = true and the worst value: an unhandled critical event cannot be erased.
Valid sample past clear threshold	CLEAR event. A non-latching rule releases; a latching one stays latched until an explicit reset (WP8 command).
Reading not OK or older than its stale limit	NOT_ELIGIBLE: no trip and no clear. A sensor fault is a fault, not an environmental alarm.
Power removed later (WP9)	Does not erase a latched real alarm.

Thresholds are not commissioned, so the rule table in `include/policy_config.h` ships with its placeholder rules **disabled**. Tests and the latency measurement enable their own rules.

7.4 Sticky store and detail queue

Two channels carry alarm information from acquisition to Safety, deliberately different:

Channel	Content	When full	Purpose
Sticky slot (one per rule)	tripped flag, first and worst value, count, first time. Updated in a portMUX section.	Cannot be full: one slot per rule, preallocated.	Guarantees Safety acts on every trip.
Detail queue (16 × AlarmEvent)	Full event: rule, decision, the 56-byte Reading.	Zero-wait send fails; an overflow counter increments.	History for reporting (WP8 journal). Losing detail never loses the alarm.

```
bool safetySubmitEvent(const AlarmEvent* e) {     // called from AcqNormal
  portENTER_CRITICAL(&s_stickyMux);
  stickyUpdate(&s_sticky[e->ruleId], e);          // a few instructions
  portEXIT_CRITICAL(&s_stickyMux);
  const bool queued = xQueueSend(s_detail, e, 0) == pdTRUE;
  if (!queued) s_detailOverflow++;                // reported by Supervisor
  xTaskNotifyGive(s_safetyTask);                  // wake Safety now
  return queued;
}
```

7.5 Writing the relay

`hal/relay.c` [PLANNED NAME] gives Safety one function, `relayWrite(bool on)`, which maps ON to the electrical level using `BOARD_RELAY1_ACTIVE_LOW` (false: active-high through the IRLZ44N / transistor stage). Safety writes only when the decided state changes and logs every change. No other module includes relay.h; a review rule enforces it.

7.6 Safety's own outputs

After each pass Safety overwrites a length-1 `SafetyStatus` [PLANNED NAME] mailbox: relay state, lamp decision and reason (dark / bright / dwell / stale / fault / interlock / disabled), latched-rule mask, overflow counter. Supervisor prints it today; BLE and MQTT will peek it in WP8.

8. Channels between tasks

IN SHORT

Every piece of data that crosses a task boundary goes through exactly one of the channels below. Each has one writer, a fixed capacity and a defined behaviour when it is full. No task ever passes another a pointer into its own memory.

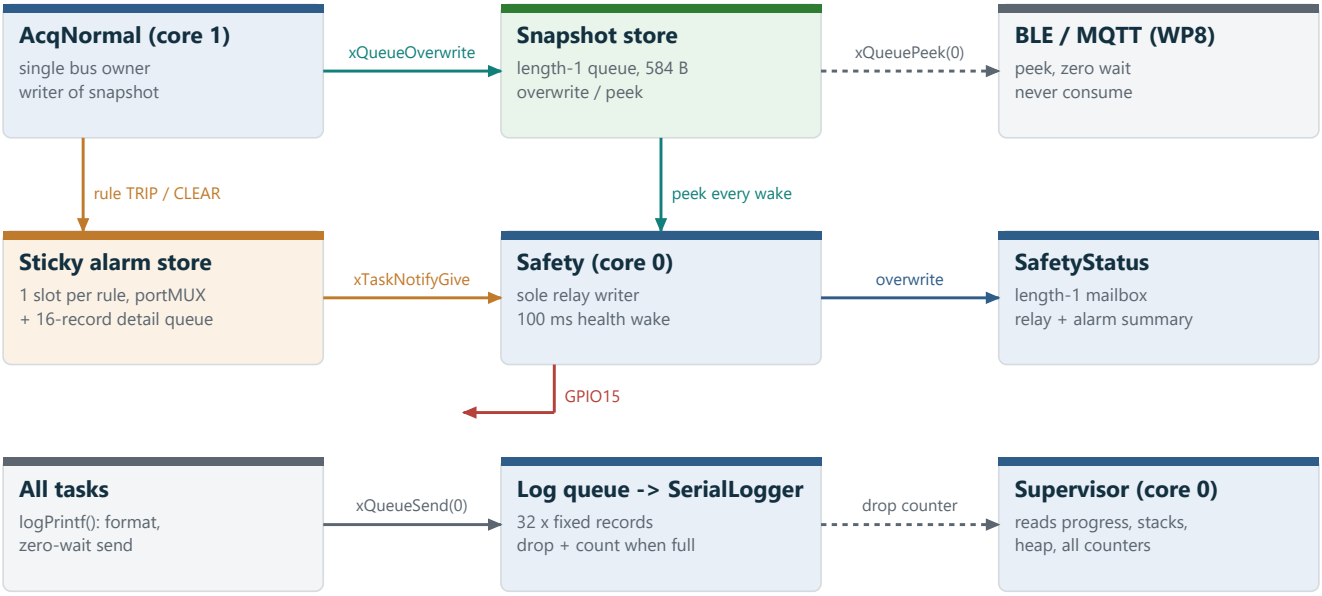


Figure 8 – Stage 4 data paths. Dashed = later readers or loss counting.

Channel	Writer → reader	Capacity	API	When full / slow reader	Status
Snapshot store	AcqNormal → Safety, Supervisor (BLE, MQTT later)	1 × 584 B	xQueueOverwrite / xQueuePeek(0)	Overwrite. Readers may skip versions by design.	[IMPLEMENTED]

Channel	Writer → reader	Capacity	API	When full / slow reader	Status
Sticky alarm slots	AcqNormal → Safety	1 per rule	portMUX section	Never full. Keeps first / worst / count.	[STAGE 4]
Alarm detail queue	AcqNormal → Safety	16 × AlarmEvent	xQueueSend(0) / Receive(0)	Drop and count overflow; sticky state unaffected.	[STAGE 4]
Safety wake	AcqNormal → Safety	counting notification	xTaskNotifyGive / ulTaskNotifyTake	Counts accumulate; one wake drains all.	[STAGE 4]
SafetyStatus	Safety → Supervisor (transports later)	1 record	xQueueOverwrite / Peek(0)	Overwrite.	[STAGE 4]
Log queue	any task → SerialLogger	32 × 129 B	xQueueSend(0) / Receive(∞)	Drop and count; the caller never blocks.	[STAGE 4]
Progress stamps	each task → Supervisor	1 × 8 B per task	portMUX section	Overwrite.	[STAGE 4]
Rail state	power ISR → all	{valid, epoch}	portMUX (ISR variant)	–	[IMPLEMENTED] record, [LATER] ISR
Command ingress, journal, lease	transports / PowerCoordinator	8 / 64 / 1	queues	Reject busy / drop oldest / one grant	[LATER]

Invariants a reviewer checks

- Every send from AcqNormal or Safety uses a **zero** timeout. A full channel is counted, never waited on.
- Only queues of plain fixed-size structs. No pointers inside queued items, no String, no JSON.
- A portMUX section contains assignments only – no call that can block, print or loop over a bus.
- One writer per channel. A second writer is a design change, not a bug fix.

9. Interrupts

> **IN SHORT**
> No sensor can raise an interrupt when its value becomes critical (§6), so alarms come from sampling, not from interrupts. Interrupts are used for timing: the HC-SR04 echo edges today, and the external-power edges in WP9.

Source	Pin / core	Handler does	Then	Status
HC-SR04 ECHO, CHANGE	GPIO18, core 1 (installed by AcqNormal)	IRAM: reads the level by register, stores an esp_timer µs timestamp for the rising and the falling edge.	AcqNormal's next poll() computes the width. No notification: the poll is already scheduled 1 ms ahead.	[IMPLEMENTED]
External power sense, CHANGE	GPIO8, core of the GPIO ISR service (see below)	IRAM: railStateInvalidateFromIsr() – valid = false, epoch++; disable itself; notify PowerCoordinator.	20 ms qualification in the task, then mode change (§13).	[LATER] WP9; needs IRAM_ATTR (audit L3)
Timer / tick, IPC, radio	SDK	Kernel and SDK internals.	–	SDK

Interactions to respect

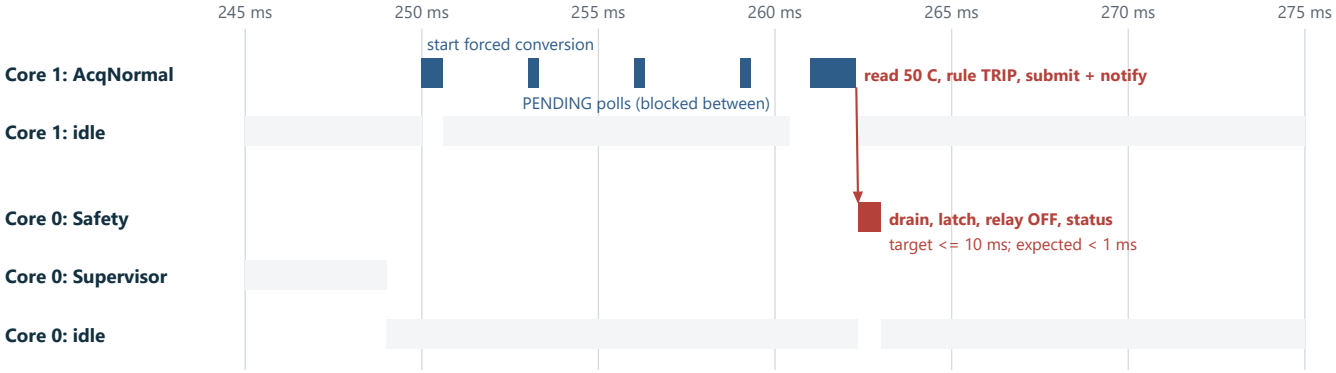
- **1-Wire masks interrupts.** Each bit slot of the bit-banged 1-Wire bus runs with interrupts off for up to ~70 µs on core 1. An echo edge arriving then is timestamped late (~1 cm). The schedule therefore never runs a DS18B20 transaction while an echo is in flight (section 6.3).
- **Where an ISR runs.** In Arduino 2.0.17 the *first* attachInterrupt call installs the shared GPIO ISR service (gpio_install_isr_service) on the calling task's core, and every later GPIO handler is dispatched from that same interrupt, on that core. AcqNormal attaches the echo ISR, so today all GPIO interrupts run on core 1. WP9 must therefore decide deliberately: either install the service from core 0 at boot (before acquisition starts) so the power-sense edge is serviced on core 0, or accept core 1 and measure the latency. Verify on hardware either way.
- **Flash writes delay GPIO interrupts.** The service is not IRAM-flagged, so during an NVS or OTA flash write (WP8) GPIO edges wait until the write finishes. An echo measured across a flash write is wrong; NVS writes must not overlap an HC-SR04 ping (Configuration and acquisition are on different cores, so WP8 needs a rule or a measurement here).
- **Interrupt watchdog.** Any ISR or critical section longer than 300 ms resets the chip. Ours are microseconds.

10. Timing, latency and CPU load

IN SHORT

The design targets three numbers from §19: ≤ 5 ms release jitter of the one-second frame, ≤ 10 ms from an accepted sample to Safety's action, and ≤ 300 ms from a sustained over-temperature to that action. The worst case is dominated by waiting for the next BME280 sample (250 ms); the task-to-task path itself is microseconds.

10.1 From a sample to the relay



Illustration, not a trace: the event crosses cores through a task notification; Safety (priority 6) preempts anything lower on core 0.

Figure 9 – A BME280 sample that trips a rule, zoomed to 245–275 ms. Both cores shown.

Segment	Mechanism	Budget	Expected
Condition → next sample	BME280 has no alert pin; sampled every 250 ms	≤ 250 ms	0–250 ms, uniform
Slot release jitter	AcqNormal wakes from its notify-wait; nothing higher on core 1	≤ 5 ms	≈ 1 ms (tick granularity)
Conversion + read	forced mode, oversampling ×1	≤ 12 ms	11 ms measured
Rule + submit + notify	pure function, portMUX update, queue copy, xTaskNotifyGive	–	a few µs
Safety wake on core 0	priority 6 preempts Supervisor; idle otherwise	≤ 10 ms incl. action	< 1 ms without radios
Total, sustained condition		≤ 300 ms (277 ms budget)	≈ 262 ms worst case

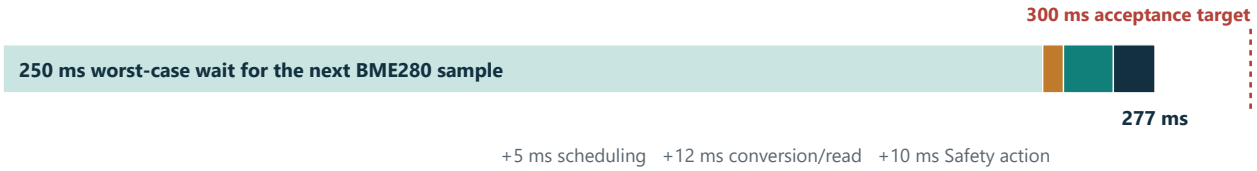


Figure 10 – Rev 3 latency budget (§6): 250 + 5 + 12 + 10 = 277 ms, accepted at 300 ms.

10.2 Light, lamp and staleness timing

Path	Time
New light reading visible to Safety	≤ 100 ms after publish (next health wake); no event is sent for the lamp.
Darkness → lamp ON	light window (≈ 99 ms) + ≤ 100 ms + the remaining dwell (up to LAMP_MIN_OFF_MS).
Acquisition stalls → lamp OFF	3 s stale limit + ≤ 100 ms.
Light driver faults → lamp OFF	POLICY_LAMP_FAULTS_TO_OFF readings, i.e. ≈ 5 s with the placeholder value of 5.

10.3 When the radios arrive (D14)

In WP8 the Wi-Fi, lwIP (priority 18) and NimBLE host tasks join core 0 and outrank Safety. Their bursts can delay Safety by more than the 10 ms target. Rev 3 therefore requires the alarm test to be repeated under continuous MQTT publishing and BLE streaming. **Fallback if it fails:** run Safety on core 1 in normal mode at a priority above AcqNormal, and hand it to core 0 only in low power using the same quiesce / grant protocol as acquisition.

10.4 CPU load

Per second, AcqNormal performs roughly 4 BME280 cycles, one DS18B20 read + start (≈ 19 ms blocking, the largest item), two SGP30 transactions, one HC-SR04 ping and 35 ADC taps. Estimated busy time is 25–35 ms per second on core 1 (≈ 3 %), against the 150 ms reservation of §9. Safety runs ten short passes per second. These are estimates to be replaced by a trace (section 16).

11. Logging, supervision and watchdogs (WP10)

IN SHORT

Logging becomes a bounded queue drained by its own low-priority task, so a slow UART never delays a sensor or Safety. Each important task stamps a progress time after real work; Supervisor checks those stamps every second and reports a stall before the 5 s task watchdog resets the chip. Each task feeds the watchdog only for itself.

11.1 Logger

Aspect	Design
Record	{ LogLevel level; char text[LOG_TEXT_LEN = 128]; } – 129 bytes, copied into the queue.
logPrintf()	vsnprintf into a local buffer, truncate at 127 characters, xQueueSend with zero wait. On failure increment the drop counter. Never blocks; never called from an ISR.
Before the task exists	Falls back to a direct printf, so boot messages before step 7 still appear.
SerialLogger task	core 1, priority 1: xQueueReceive(portMAX_DELAY), then printf('[%c] %s'). It only runs when AcqNormal is Blocked.
Bounds	Level checked before indexing the tag table (audit D8).
Stack note	vsnprintf with %f uses about 1 KiB of stack in the caller. Safety (3 KiB) and Supervisor log floats; their high-water marks are checked in the acceptance run.

11.2 Progress stamps

`progressMark(TASK_ID_x)` stores `monoNowMs()` in a per-task 64-bit slot under a portMUX section (a 64-bit value cannot be written atomically, section 2.6). It is called after *meaningful* work: AcqNormal after a pass that advanced at least one job, Safety after each completed pass. A task blocked forever stops stamping, even though the CPU and the idle task are fine.

11.3 Supervisor

Every	Check	Action
1 s	Progress age: Safety > 1 s, AcqNormal > 2 s, SerialLogger > 2 s while its queue is non-empty	WARN once per episode, set degraded status
1 s	Counters: alarm-detail overflow, log drops, acquisition late / skipped, driver faults	Log changes only
5 s	Status line: key readings, health, lamp state and reason, jitter min / max	INFO
10 s	uxTaskGetStackHighWaterMark per task, esp_get_minimum_free_heap_size, largest free internal block	WARN below 512 B of stack or a falling heap trend
boot	esp_reset_reason()	INFO; a TASK_WDT or PANIC reason is a WARN

11.4 Task watchdog

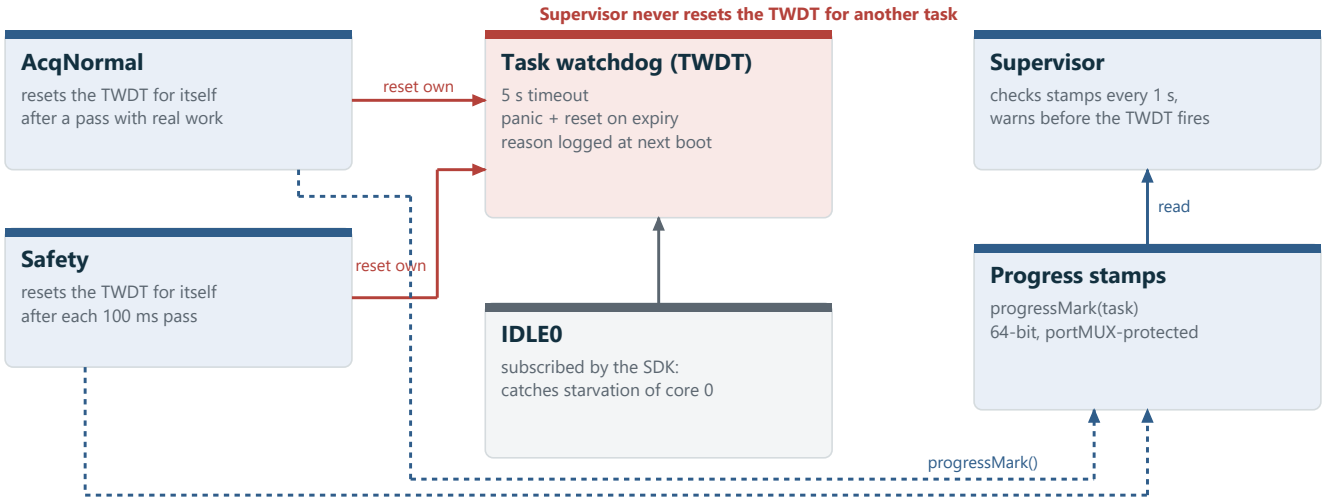


Figure 11 – Who feeds what. Each subscribed task resets the TWDT only for itself.

- The Arduino core has already initialised the TWDT (5 s, panic). Tasks only call `esp_task_wdt_add(NULL)` once at start and `esp_task_wdt_reset()` after meaningful work.
- Subscribed in Stage 4: **AcqNormal** and **Safety**. Not subscribed: SerialLogger and Supervisor (lowest priority; legitimately starved under load – their stalls are logged, not fatal).
- Core 1 starvation is *not* covered by the SDK (IDLE1 unsubscribed). AcqNormal's own subscription covers the case that matters.
- Supervisor never resets the watchdog on behalf of another task: that would hide exactly the stall the watchdog exists to catch (§18).
- WP9 adds the rule from §18: an inactive acquisition worker blocks forever by design and must unsubscribe before it blocks, as part of the lease handover.

WORKED EXAMPLE – INJECTED ACQUISITION STALL

A test build (`-D FW_INJECT_STALL_ACQ`) makes AcqNormal block forever after 60 s. At ~61 s Safety's freshness check sees light data older than 3 s → lamp OFF. At ~62 s Supervisor logs `WARN acq: no progress for 2003 ms`. At ~65 s the TWDT expires for AcqNormal → panic → reset. After the reboot Supervisor logs `reset reason: TASK_WDT`.

12. Memory

IN SHORT

Everything the RTOS design needs is allocated statically at boot, in internal SRAM: about 21 KiB for Stage 4. Nothing is allocated after setup(), so heap fragmentation cannot build up over days of running.

Item	Size	Where	Status
Safety stack + TCB	3 KiB + ~350 B	static	[STAGE 4]
AcqNormal stack + TCB	4 KiB + ~350 B	static	[STAGE 4]
SerialLogger stack + TCB	2 KiB + ~350 B	static	[STAGE 4]
Supervisor stack + TCB	3 KiB + ~350 B	static	[STAGE 4]
Snapshot store	584 B + queue header	static	[IMPLEMENTED]
Working snapshot (AcqNormal)	584 B	static	[STAGE 4]
Safety snapshot copy, Supervisor snapshot copy	2 × 584 B	static (not on the stack)	[STAGE 4]
Alarm detail queue	16 × 60 B ≈ 960 B	static	[STAGE 4]
Log queue	32 × 129 B ≈ 4.1 KiB	static	[STAGE 4]
Sticky alarm slots, SafetyStatus, progress	< 0.5 KiB	static	[STAGE 4]
Driver contexts and HAL structs	existing	static	[IMPLEMENTED]
Total added by Stage 4	≈ 21 KiB	of ~320 KiB free internal RAM	

The production image used 18.9 KB of static RAM before Stage 4 (audit, 25 Oct). Stack sizes are Rev 3 starting budgets: they are confirmed or corrected from the measured high-water marks (section 11.3), keeping at least 25 % headroom. PSRAM is not used for anything in this design.

13. Failure behaviour

IN SHORT

Every failure has a defined, bounded behaviour: mark the data, keep sensing and Safety running, report, retry later; reset only as a last resort with the reason logged (§18).

Failure	Detected by	Behaviour	Lamp
Sensor missing at boot	init() fails	Reading STARTING / FAULT; init retried at a bounded interval	unaffected (unless light)
Light sensor faulty	driver FAULT	Fault counter; after N readings lamp OFF	OFF
Light wire pulled	not detectable (reads dark)	Lamp may turn ON – hardware decision needed	may turn ON
I2C bus stuck low	transaction timeout + SDA low	Bus recovery once, both I2C readings FAULT meanwhile	unaffected
DS18B20 no presence	reset without presence	FAULT written (audit L2); TDS becomes DEPENDENCY_INVALID	unaffected
Deadline overrun	schedule lateness	kQualityLate, skip missed slots, count	unaffected
Alarm detail queue full	send fails	Overflow counted; sticky slot still triggers Safety	interlock still applies
Log queue full	send fails	Drop and count; no task blocks	unaffected
AcqNormal stalls	progress age; data age	Lamp OFF at 3 s, WARN at 2 s, TWDT reset at 5 s	OFF
Safety stalls	progress age	WARN at 1 s, TWDT reset at 5 s; relay keeps its last level until reset, then OFF	last state ≤ 5 s
Stack overflow	canary at context switch	Panic and reset; reason logged on next boot	OFF after reset
Init of a service fails	returns false	ERROR logged, boot continues, relay stays OFF	OFF

14. What changes in WP8 and WP9

IN SHORT

Stage 4 is built so that the later work packages add tasks and protocol without rewriting it. WP8 adds readers and a command path; WP9 adds a second acquisition worker on core 0 and moves the sensors between the two with a generation-numbered lease.

14.1 WP8 – transports and configuration

- WifiMqtt and BleTransport peek the snapshot and SafetyStatus with zero wait; they never call a driver.
- Commands go through an 8-entry ingress queue to Configuration (NVS owner) or, for actuation, to Safety. Safety checks interlocks and expiry before applying any of them (§12 step 4).
- The alarm detail queue gains a 64-event journal with independent cursors for BLE and MQTT.
- The SGP30 baseline and the lamp thresholds become NVS-backed configuration with a revision number (already carried in every Reading).
- D14 radio-load test for Safety (section 10.3).

14.2 WP9 – power modes and the lease

The external-power sense on GPIO8 (measured 25 Oct) drives the mode machine. The ISR invalidates the 5 V rail and bumps its epoch; PowerCoordinator qualifies the edge for 20 ms and runs the transitions below. Acquisition's notify-wait (section 6.5) is what lets PowerCoordinator interrupt it at once.

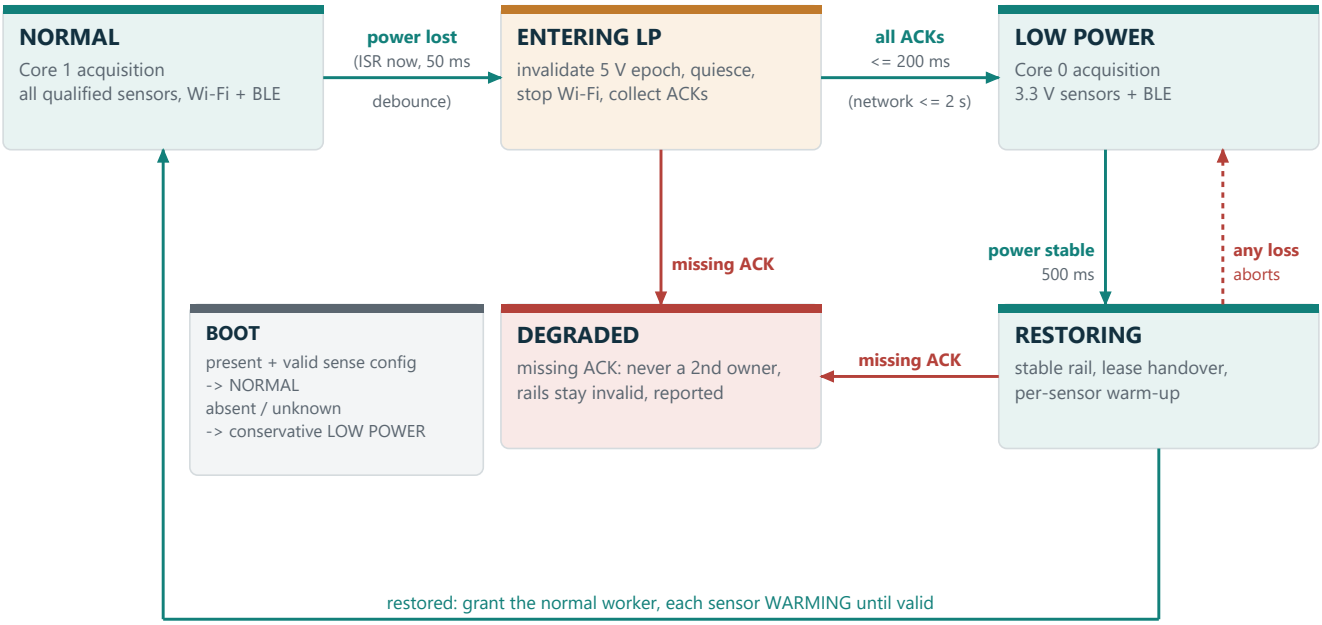


Figure 12 – Power modes (Rev 3 §13).

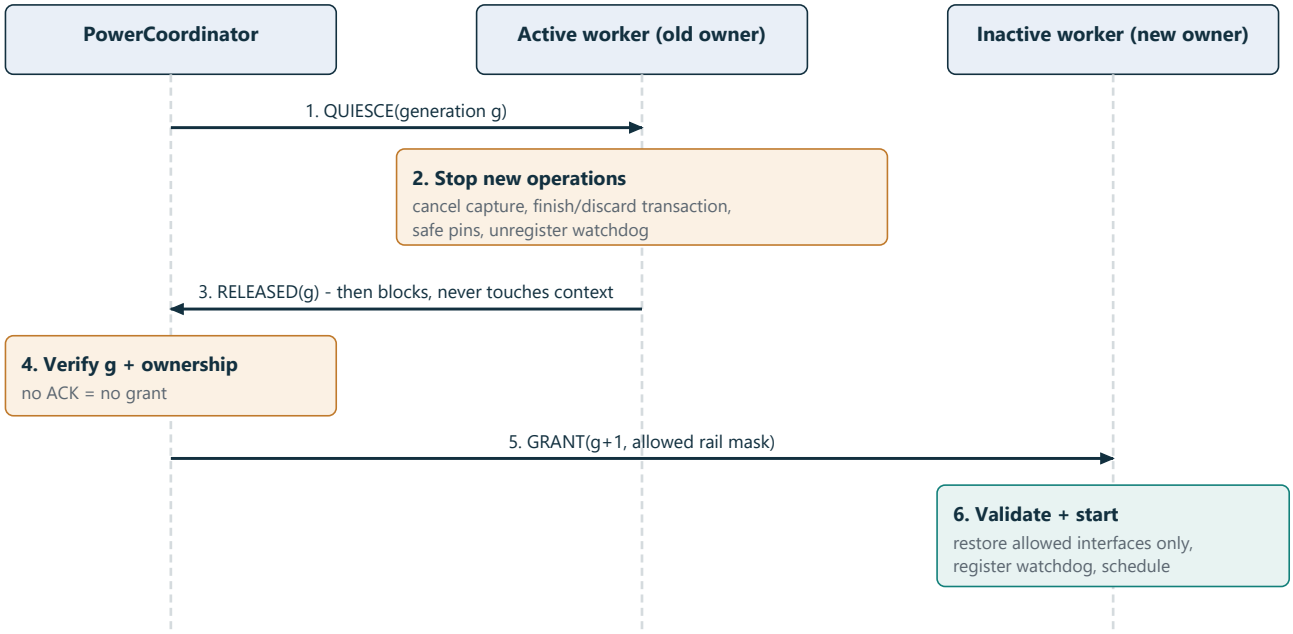


Figure 13 – Lease handover between the two acquisition workers (Rev 3 §14).

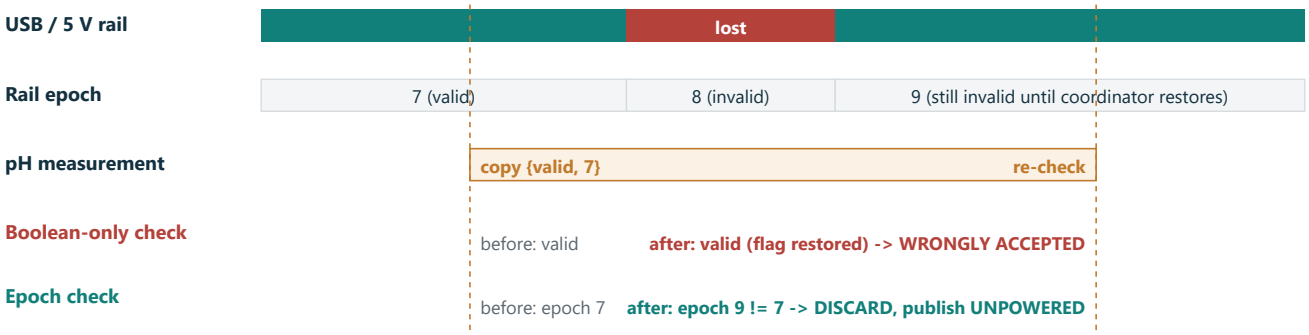


Figure 14 – Why each 5 V reading carries the rail epoch it started in (Rev 3 §14).

Prepared in Stage 4 so WP9 needs no rework: the engine and job table are worker-agnostic; every Reading has a powerEpoch field; drivers already implement cancel() and preparePowerOff(); the task watchdog subscription is per task, so a worker can unsubscribe when it releases the lease.

15. Implementation plan and file map

IN SHORT

Four steps, each ending in a test: audit fixes, pure domain logic with host tests, the FreeRTOS services on target, then diagnostics. Frozen headers are not edited; new tunables go into a new, non-frozen header.

Step	Content	Ends with
1 Audit prerequisites	L1 evaluateRule signature; L2 DS18B20 NOT_PRESENT → FAULT; L4 safe GPIO defaults; L5 recover I2C only when SDA is low; D1 test fix; D2 boot wording; D3 comment; D6 lastGoodMs meaning (reading.h wording → team-lead review); D7 snapshotReset identity; D8 logger bounds.	pio test -e native, all envs build
2 Domain logic	alarm.c (hysteresis, persistence, latch), schedule.c (deadlines, skip, BME280 fit), safety_core.c (§12 order, lamp, staleness, interlock), lampControlForceOff(), policy_config.h.	New host tests pass, 0 warnings
3 Services (WP7)	acquisition.c engine with all drivers, safety.c task + sticky store + detail queue, hal/relay.c, main.c boot order.	Lamp follows light on the bench
4 Diagnostics (WP10)	logger queue + SerialLogger, progress portMUX, supervisor.c, TWDT subscriptions.	Stall injection passes

File map

File	Role	Status
include/app_config.h	Periods, budgets, queue lengths, cfgTaskSpec() – FROZEN	[IMPLEMENTED]
include/board_profile.h	Pins, relay polarity, sense gate – FROZEN	[IMPLEMENTED]
include/policy_config.h	Lamp thresholds and dwell, auto-enable switch, alarm rule table (disabled)	[IMPLEMENTED]
src/main.c	Boot order only	[IMPLEMENTED]
src/domain/alarm.c/.h	Rule evaluation, stale limits	[IMPLEMENTED]
src/domain/schedule.c/.h	Job table, deadlines, skip / late accounting	[IMPLEMENTED]
src/domain/safety_core.c/.h	Pure Safety state machine	[IMPLEMENTED]
src/domain/lamp_control.c/.h	Lamp hysteresis and dwell	[IMPLEMENTED]
src/services/acquisition.c/.h	AcqNormal task and engine	[IMPLEMENTED]
src/services/safety.c/.h	Safety task, sticky store, detail queue, SafetyStatus	[IMPLEMENTED]
src/services/snapshot_store.c/.h	Length-1 snapshot queue	[IMPLEMENTED]
src/services/supervisor.c/.h	Supervisor task	[IMPLEMENTED]
src/services/rail_state.c/.h, power_coordinator.c/.h	Rail epoch, power modes	[IMPLEMENTED] record / [LATER] WP9
src/hal/relay.c/.h	relayWrite(); included by Safety only	[IMPLEMENTED]
src/hal/gpio_safe.c	Boot-time safe levels	[IMPLEMENTED]
src/diagnostics/logger.c, progress.c	Log queue + SerialLogger, progress stamps	[IMPLEMENTED]
test/native/test_domain/test_main.c	Host tests for all domain logic	[IMPLEMENTED] + new tests
test/bringup/*_main.c	Per-sensor bring-up envs (unchanged; test_lamp superseded by production)	[IMPLEMENTED]

16. How we prove it works

IN SHORT

Host tests prove the logic with injected time; bench runs prove the timing and the relay on real hardware. Each line maps to a row of the §19 acceptance table.

16.1 Host tests (pio test -e native)

Area	Test cases
alarm	trip on one sample with persist 0; trip only after persistMs; no chatter inside hysteresis; stale sample NOT_ELIGIBLE; FAULT NOT_ELIGIBLE; latch survives clear until reset
schedule	BME280 due 4× per 1 s; DS18B20 due once; after a 3 s gap each job runs once and is counted late/skipped (no burst); a 50 ms step is deferred past a BME280 slot 20 ms away
safety_core	boot OFF; dark → ON only after the dwell; stale light (no new sample) → OFF within 3 s + one wake; N FAULT samples → OFF; interlock beats the lamp; later normal sample does not erase an unhandled TRIP; auto disabled → relay never ON
existing	the 23 current tests keep passing; D1 made non-vacuous

16.2 On the board (pio run -e esp32dev -t upload -t monitor)

Test (§19 row)	Procedure	Pass when
Build	pio run for esp32dev and all 11 test_* envs	all SUCCESS, RAM / flash recorded in README
One-second schedule	Supervisor logs frame release jitter min / max for 30 min	max ≤ 5 ms; every metric updates ~1 s, BME280 ~250 ms
Temperature alarm	Test build enables a rule that trips at the current air temperature; log sample-end and Safety-action times at varied phases	sample → action ≤ 10 ms; condition → action ≤ 300 ms
Lamp	Low-voltage load on COM/NO first: cover sensor; torch; hold inside the band; reset	ON after dwell; OFF after dwell; no chatter; OFF through boot
Stall injection	FW_INJECT_STALL_ACQ, then one with a stalled Safety	lamp OFF ≤ 3.1 s; WARN; TWDT reset; reason logged
Queue overload	Test rule that trips every sample with a 1-entry detail queue	overflow counted, Safety still latches every time
Endurance	24 h run	no unexpected reset, flat heap and stack high-water marks, counters plausible

17. Rules for everyone writing firmware

IN SHORT

A checklist for code review. Breaking one of these is a design change, discussed first – not a shortcut.

Do	Do not
Call drivers and bus HALs only from the acquisition engine.	Read a sensor from Safety, a transport, a test hook or loop().
Write GPIO15 only through relayWrite() in Safety.	Include hal/relay.h anywhere else, or drive the pin from a bring-up main in production.
Block on a notification or queue with a bounded timeout.	vTaskDelay(1000) in a production task, busy-wait, or poll a flag in a loop.
Send with a zero timeout and count failures.	Wait on a full queue from AcqNormal or Safety.
Pass fixed-size structs by value through queues.	Pass a pointer to a stack buffer, a String or a JSON object to another task.
Protect cross-core multi-field updates with portMUX; keep the section to a few assignments.	Rely on volatile for cross-core consistency, or call anything that can block inside a critical section.
Put decisions in src/domain with now as a parameter, and test them on the host.	Include Arduino.h or FreeRTOS headers in src/domain.
Allocate tasks, queues and buffers statically at boot.	Call malloc after setup(); keep a Snapshot on a small task stack.
Use monoNowMs() for every age and deadline.	Use millis() differences across tasks, tick counts as time, or wall-clock time.
Log with logPrintf() (non-blocking).	Call printf from production tasks or from an ISR.
Reset the task watchdog only from the task itself, after real work.	Feed the watchdog for another task, or from a timer.
Take task names, cores, priorities and stacks from cfgTaskSpec().	Hard-code a priority or core in xTaskCreate.
Report missing data as a Health value.	Publish 0 for a value you do not have.

Glossary

Term	Meaning here
Frame	The one-second acquisition cycle that starts at each whole second of the schedule.
Slot	One BME280 run of the 250 ms fast lane.
Job / DriverRun	A schedule entry for one driver and its init → start → poll stepping state.
Snapshot	The latest copy of all readings, 584 B, published by acquisition, peeked by readers.
Sticky slot	Per-rule alarm record that cannot be lost when queues are full.
Stale	A reading older than its limit: 1 s for the BME280, 3 s for the others.
Dwell	Minimum time the lamp stays in a state after a change.
Interlock	A latched alarm whose action overrides normal control (relay forced OFF).
Progress stamp	Last time a task did meaningful work; read by Supervisor.
Epoch / lease	WP9: rail-change counter in each 5 V reading; exclusive right of one acquisition worker to the buses.